

UNIVERSITATEA DE STAT
Facultatea Matematică și Informatică
Specialitatea Informatică
Disciplina: Automatizare și scriptare

DARE DE SEAMĂ
LUCRAREA DE LABORATOR NR. 2
Script Python pentru obținerea cursului valutar prin API

A elaborat: Pisari Adriana
Grupa: I2401
An de studii: III

Chișinău, 2026

1. Scopul lucrării

Scopul lucrării este realizarea unui script Python care comunică cu un serviciu API local pentru obținerea cursului de schimb dintre două valute la o dată specificată. Scriptul primește valutele și data din linia de comandă, trimite cererea către API, salvează răspunsul în format JSON și gestionează eventualele erori prin mesaje clare în consolă și într-un fișier de log.

În cadrul lucrării au fost urmărite următoarele obiective:

- crearea și utilizarea branch-ului Git lab02;
- organizarea fișierelor în directorul lab02;
- utilizarea parametrilor din linia de comandă;
- trimiterea cererilor HTTP către un API local;
- salvarea rezultatelor în fișiere JSON în directorul data;
- gestionarea și jurnalizarea erorilor în error.log;
- utilizarea unui mediu virtual Python și a fișierului requirements.txt;
- testarea scriptului pe minimum 5 date la intervale egale.

2. Pregătirea și verificarea serviciului API

Serviciul suport primit în arhiva laboratorului rulează local prin Docker și este accesibil la adresa <http://localhost:8080>. Înainte de scrierea scriptului Python am verificat manual că API-ul răspunde corect.

2.1. Verificarea listelor de valute

```
curl "http://localhost:8080/?currencies" -X POST -d "key=EXAMPLE_API_KEY"
```

Comanda curl trimite o cerere HTTP către serviciul local. Opțiunea -X POST stabilește metoda POST, iar -d trimite cheia API în corpul cererii. Răspunsul conține câmpurile error și data; un câmp error gol înseamnă că solicitarea a fost executată fără eroare.

```
[adriana@Adrianas-MacBook-Air lab02prep % curl "http://localhost:8080/?currencies" -X POST -d "key=EXAMPLE_API_KEY"]  
{"error":"","data":["MDL","USD","EUR","RON","RUS","UAH"]}%  
adriana@Adrianas-MacBook-Air lab02prep %
```

2.2. Testarea manuală a unei conversii

```
curl "http://localhost:8080/?from=USD&to=EUR&date=2025-01-01" -X POST -d "key=EXAMPLE_API_KEY"
```

Parametrul from indică valuta sursă, to indică valuta destinație, iar date indică data pentru care este solicitat cursul. Răspunsul API include from, to, rate și date. Acest test a confirmat că endpoint-ul de conversie funcționează înainte de integrarea sa în scriptul Python.

```
adriana@Adrianas-MacBook-Air lab02prep % curl "http://localhost:8080/?from=USD&to=EUR&date=2025-01-01" -X POST -d "key=EXAMPLE_API_KEY"
{"error":"","data":{"from":"USD","to":"EUR","rate":1.0449967801462194,"date":"2025-01-01"}}%
```

3. Pregătirea repository-ului Git

3.1. `cd ~/Desktop/automation`

```
cd ~/Desktop/automation
```

Schimbă directorul curent în repository-ul local automation.

3.2. `pwd`

```
pwd
```

Afișează calea completă a directorului curent și confirmă că lucrăm în repository-ul corect.

3.3. `git status`

```
git status
```

Afișează branch-ul curent și starea fișierelor din repository.

3.4. `git checkout main`

```
git checkout main
```

Comută pe branch-ul principal main.

3.5. `git pull`

```
git pull
```

Actualizează copia locală cu modificările existente pe repository-ul remote.

3.6. `git checkout -b lab02`

```
git checkout -b lab02
```

Creează branch-ul lab02 și comută imediat pe acesta.

3.7. `git branch`

```
git branch
```

Listează branch-urile locale; caracterul * indică branch-ul activ.

3.8. mkdir lab02

```
mkdir lab02
```

Creează directorul lab02 cerut de sarcină.

3.9. cd lab02

```
cd lab02
```

Intră în directorul lab02.

3.10. nano currency_exchange_rate.py

```
nano currency_exchange_rate.py
```

Creează/deschide scriptul Python în editorul nano.

```
adriana@Adrianas-MacBook-Air lab02 % cat currency_exchange_rate.py

import argparse
import json
import logging
import os
from datetime import datetime
from pathlib import Path

import requests

API_URL = "http://localhost:8080"
MIN_DATE = datetime.strptime("2025-01-01", "%Y-%m-%d").date()
MAX_DATE = datetime.strptime("2025-09-15", "%Y-%m-%d").date()

# lab02/ este folderul scriptului.
SCRIPT_DIR = Path(__file__).resolve().parent

# Directorul părinte este rădăcina repository-ului automation.
PROJECT_ROOT = SCRIPT_DIR.parent

# data trebuie să fie în root-ul proiectului.
DATA_DIR = PROJECT_ROOT / "data"

# error.log trebuie să fie tot în root-ul proiectului.
LOG_FILE = PROJECT_ROOT / "error.log"

logging.basicConfig(
    filename=LOG_FILE,
    level=logging.ERROR,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

def validate_date(date_string):
    """
    Verifică dacă data este în format YYYY-MM-DD
    și dacă aparține perioadei permise.
    """
    try:
        request_date = datetime.strptime(
            date_string,
```

4. Alegerea structurii scriptului

Pentru script am ales o structură modulară, împărțită în funcții, astfel încât fiecare funcție să aibă o singură responsabilitate. Această abordare face codul mai ușor de înțeles, testat și modificat.

Componentă	Rol
argparse	Preia valutele și data din linia de comandă.
requests	Trimite cererea HTTP POST către API.
json	Salvează răspunsul API în fișier JSON.
logging	Scrie erorile în error.log.
datetime	Validează formatul și intervalul datei.
pathlib.Path	Construiește căile către data și error.log independent de directorul din care este lansat scriptul.
validate_date()	Verifică formatul YYYY-MM-DD și intervalul 2025-01-01 – 2025-09-15.
get_exchange_rate()	Construiește și trimite cererea către API și verifică răspunsul.
save_to_json()	Creează directorul data dacă este necesar și salvează rezultatul.
main()	Coordonează citirea argumentelor, validarea, request-ul, salvarea și tratarea erorilor.

4.1. Preluarea argumentelor din Terminal

```
python3 currency_exchange_rate.py USD EUR 2025-01-01
```

Cele trei valori sunt citite prin argparse: USD devine valuta sursă, EUR valuta destinație, iar 2025-01-01 data cererii. Alegerea parametrilor din linia de comandă permite reutilizarea aceluiași script fără modificarea codului.

4.2. Validarea datei

Funcția `validate_date()` utilizează `datetime.strptime(..., "%Y-%m-%d")` pentru a verifica formatul. În plus, verifică dacă data se află în perioada permisă de setul de date: 2025-01-01 – 2025-09-15. Dacă validarea eșuează, este generată o eroare clară.

4.3. Cererea către API

```
response = requests.post(API_URL, params=params, data=post_data, timeout=10)
```

requests.post() trimite parametrii from, to și date în URL, iar cheia API în corpul cererii. timeout=10 împiedică scriptul să aștepte nelimitat în cazul în care serviciul nu răspunde.

4.4. Salvarea JSON

```
DATA_DIR.mkdir(exist_ok=True)
```

Comanda creează directorul data dacă acesta nu există. Parametrul exist_ok=True permite rulări repetate fără eroare dacă directorul a fost deja creat.

```
json.dump(data, json_file, indent=4, ensure_ascii=False)
```

Scrie răspunsul API într-un fișier JSON lizibil. Numele fișierului include cele două valute și data, de exemplu USD_EUR_2025-01-01.json.

4.5. Gestionarea erorilor

Scriptul tratează separat probleme precum lipsa conexiunii la API, timeout-ul, alte erori HTTP, răspunsul JSON invalid, valuta necunoscută și data invalidă. Eroarea este afișată în consolă și este înregistrată în error.log prin modulul logging.

5. Configurarea mediului Python

La prima execuție a scriptului a apărut eroarea ModuleNotFoundError deoarece biblioteca requests nu era instalată.

```
python3 currency_exchange_rate.py USD EUR 2025-01-01
```

```
main /
[adriana@Adrianas-MacBook-Air lab02 % python3 currency_exchange_rate.py USD EUR 2
025-01-01
Traceback (most recent call last):
  File "/Users/adriana/Desktop/automation/lab02/currency_exchange_rate.py", line
  8, in <module>
    import requests
ModuleNotFoundError: No module named 'requests'
```

Încercarea de instalare globală cu pip a fost blocată de mediul Python administrat de Homebrew pe macOS.

```
python3 -m pip install requests
```

```
import requests
ModuleNotFoundError: No module named 'requests'
[adriana@Adrianas-MacBook-Air lab02 % python3 -m pip install requests
error: externally-managed-environment
```

Soluția aleasă a fost utilizarea unui mediu virtual Python, ceea ce izolează dependențele proiectului de instalarea globală.

```
python3 -m venv venv
```

Creează mediul virtual în directorul venv.

```
source venv/bin/activate
```

Activează mediul virtual pe macOS; prefixul (venv) din Terminal confirmă activarea.

```
python3 -m pip install requests
```

Instalează biblioteca requests în mediul virtual, fără a modifica Python-ul global.

```
python3 -c "import requests; print(requests.__version__)"
```

Verifică faptul că requests poate fi importată și afișează versiunea instalată.

```
requests.__version__  
2.34.2
```

6. Prima rulare reușită

```
python3 currency_exchange_rate.py USD EUR 2025-01-01
```

După activarea mediului virtual, scriptul a executat cererea cu succes, a afișat cursul USD -> EUR și a salvat rezultatul în directorul data din rădăcina repository-ului.

Cererea a fost realizată cu succes.

USD -> EUR

Data: 2025-01-01

Curs: 1.0449967801462194

Date salvate în: /Users/adriana/Desktop/automation/data/USD_EUR_2025-01-01.json

7. Fișiere auxiliare pentru reproducerea mediului

7.1. requirements.txt

```
pip freeze > requirements.txt
```

pip freeze listează pachetele instalate în mediul virtual, iar operatorul > redirecționează lista în requirements.txt. Acest fișier permite reinstalarea dependențelor cu pip install -r requirements.txt.

```
cat requirements.txt
```

Afișează conținutul fișierului pentru verificare. În el apar requests și dependențele sale.

```
[(venv) adriana@Adrianas-MacBook-Air lab02 % cat requirements.txt  
certifi==2026.7.22  
charset-normalizer==3.5.1  
idna==3.20  
requests==2.34.2  
urllib3==2.8.0
```

7.2. .gitignore

```
nano .gitignore
```

Fișierul .gitignore a fost creat în rădăcina repository-ului pentru a exclude fișierele care nu trebuie încărcate pe GitHub.

```
venv/  
__pycache__/  
*.pyc  
.DS_Store
```

Astfel, mediul virtual, cache-ul Python și fișierele specifice macOS sunt ignorate de Git.

8. Testarea pe 5 date la intervale egale

Conform cerinței, scriptul a fost rulat pentru cinci date cu intervale egale de 60 de zile, folosind perechea USD/EUR.

```
python3 lab02/currency_exchange_rate.py USD EUR 2025-01-01  
python3 lab02/currency_exchange_rate.py USD EUR 2025-03-02  
python3 lab02/currency_exchange_rate.py USD EUR 2025-05-01  
python3 lab02/currency_exchange_rate.py USD EUR 2025-06-30  
python3 lab02/currency_exchange_rate.py USD EUR 2025-08-29
```

Fiecare execuție a trimis o cerere separată către API și a creat câte un fișier JSON în directorul data.

```
ls data
```

Comanda listează fișierele create și demonstrează că există rezultatele pentru toate cele cinci date.

```
[(venv) adriana@Adrianas-MacBook-Air automation % ls data  
USD_EUR_2025-01-01.json USD_EUR_2025-05-01.json USD_EUR_2025-08-29.json  
USD_EUR_2025-03-02.json USD_EUR_2025-06-30.json _
```

9. Testarea gestionării erorilor

9.1. Valută necunoscută

```
python3 lab02/currency_exchange_rate.py ABC EUR 2025-05-01
```

ABC nu este o valută acceptată de serviciu. Scriptul a afișat mesajul primit de la API și a înregistrat eroarea în fișierul error.log.

```
cat error.log
```

cat afișează conținutul fișierului de log, unde se poate verifica data, perechea de valute și descrierea erorii.


```
[(venv) adriana@Adrianas-MacBook-Air automation % cat error.log
2026-09-27 18:51:52,451 - ERROR - ABC -> EUR, data 2025-05-01: The currency ABC
is unknown
-----
```

9.2. Dată în afara intervalului

```
python3 lab02/currency_exchange_rate.py USD EUR 2026-01-01
```

Data 2026-01-01 este în afara perioadei acceptate. Funcția `validate_date()` oprește execuția înainte de cererea API și afișează un mesaj explicit. Eroarea este de asemenea înregistrată în `error.log`.

```
2026-09-27 18:52:10,255 - ERROR - USD -> EUR, data 2026-01-01: Data trebuie să f
ie între 2025-01-01 și 2025-09-15.
-----
```

10. Documentarea proiectului prin README.md

```
nano README.md
```

README.md din directorul lab02 descrie scopul scriptului, cerințele, instalarea mediului virtual și a dependențelor, modul de rulare, exemplele de utilizare, structura funcțiilor și comportamentul la erori. Astfel, o altă persoană poate reproduce mediul și poate utiliza scriptul fără a citi mai întâi întregul cod sursă.

11. Finalizarea în Git și încărcarea pe GitHub

După verificarea tuturor fișierelor, modificările se salvează în branch-ul lab02 și se trimit pe repository-ul remote. Apoi branch-ul se integrează în main.

```
git status
```

Verifică fișierele noi și modificate înainte de commit.

```
git add .
```

Adaugă toate modificările relevante în staging area.

```
git commit -m "Add lab02 currency exchange rate script"
```

Creează un commit cu modificările laboratorului.

```
git push -u origin lab02
```

Trimite branch-ul lab02 pe GitHub și stabilește legătura cu branch-ul remote.

```
git checkout main
```

Revine pe branch-ul principal.

```
git pull
```

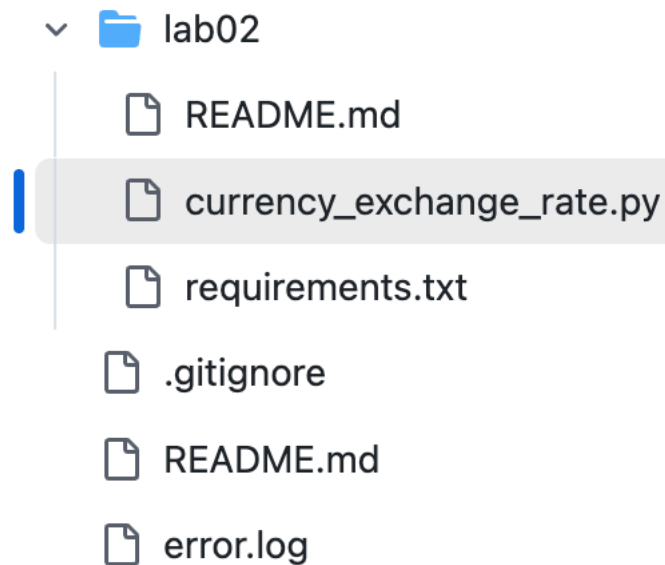
Actualizează main înainte de merge.

```
git merge lab02
```

Integrează modificările din lab02 în main.

```
git push origin main
```

Trimite pe GitHub versiunea main care conține laboratorul 2.



12. Structura finală a proiectului

```
automation/  
├── .gitignore  
├── error.log  
├── data/  
│   ├── USD_EUR_2025-01-01.json  
│   ├── USD_EUR_2025-03-02.json  
│   ├── USD_EUR_2025-05-01.json  
│   ├── USD_EUR_2025-06-30.json  
│   └── USD_EUR_2025-08-29.json  
├── lab01/  
└── lab02/  
    ├── currency_exchange_rate.py  
    ├── README.md  
    ├── requirements.txt  
    └── venv/ (local, ignorat de Git)
```

13. Concluzii

- Am învățat să trimit cereri HTTP din Python către un API folosind biblioteca requests.

- Am folosit `argparse` pentru a transmite valutele și data din linia de comandă, fără modificarea codului la fiecare rulare.
- Am implementat validarea datelor, salvarea rezultatelor în JSON și jurnalizarea erorilor în `error.log`.
- Am înțeles rolul unui mediu virtual Python și al fișierului `requirements.txt` pentru izolarea și reproducerea dependențelor.
- Am testat scriptul pe cinci date la intervale egale și am verificat comportamentul în cazuri de eroare.
- Am utilizat Git pentru a lucra într-un branch separat și pentru pregătirea proiectului în vederea încărcării pe GitHub.